

Objective

Design and implement a modular Embedded C application that simulates a simplified Vehicle ECU responsible for:

- reading validated vehicle inputs
- executing control logic in a defined order
- detecting and tracking faults
- managing state transitions
- generating structured system logs

The solution must demonstrate deterministic execution, defensive programming, structured modular design, and clear separation of interfaces and implementation, similar to real embedded software development.

Core System Expectation

The hackathon must be implemented as a cyclic ECU-style execution model, where every cycle runs tasks in a fixed order.

It must behave like a small embedded software system.

Execution Order / Scheduler Requirement

A clear execution flow is mandatory.

The application must run inside a continuous loop and execute the following functions in the same order during every cycle:

Scheduler Flow per Cycle

1. **Read Inputs**
 - Read speed, temperature, gear, and requested mode
 - Validate raw inputs
2. **Update Vehicle Mode**
 - Check mode transition legality
 - Update current mode or force FAULT mode
3. **Run Control Logic**
 - Evaluate overspeed
 - Evaluate temperature thresholds

- Evaluate gear validity
- 4. **Update Fault Manager**
 - Set/clear fault flags
 - Increment counters
 - Track persistent faults
- 5. **Evaluate System State**
 - Determine NORMAL / DEGRADED / SAFE
 - Apply escalation rules
- 6. **Generate Logs**
 - Print cycle summary
 - Print warnings, faults, state changes
- 7. **Wait for Next Cycle / Continue**
 - Next user input set or next simulation round

Mandatory Note

Learners must clearly document:

- why this order is used
- what can go wrong if the order changes

Functional Requirements

1. Input Layer

The system must accept:

- vehicle speed
- engine temperature
- gear position
- requested operating mode

Validation Rules

- Speed: 0 to 200
- Temperature: -40 to 150
- Gear: 0 to 5

- Mode: valid enum values only

If any input is invalid:

- log the issue
 - do not crash
 - preserve last known valid value where appropriate
-

2. Mode Controller

Supported modes:

- OFF
- ACC
- IGNITION_ON
- FAULT

Requirements

- implement mode logic using enum
- use switch-case for mode handling
- detect illegal transitions
- force FAULT mode on invalid transitions
- maintain previous mode and current mode

Example:

- OFF → ACC allowed
 - ACC → IGNITION_ON allowed
 - OFF → IGNITION_ON illegal unless explicitly justified
-

3. Control Logic

The ECU must evaluate:

Overspeed

- if speed > 120, trigger overspeed warning/fault logic

Temperature

- if temperature > 110, critical overheat

- else if temperature > 95, high temperature
- otherwise normal

Gear Check

- valid gears: 0 to 5
- invalid gear must raise a fault

Priority Handling

If multiple issues occur in the same cycle, learners must define and document a **priority order** for reporting.

Suggested priority:

1. Critical Overheat
 2. Invalid Mode / Gear
 3. Overspeed
 4. High Temperature
-

4. Fault Manager

Faults must be managed using:

- bitwise fault flags
- fault counters
- persistent fault tracking across cycles

Minimum fault types

- overspeed
- overtemperature
- invalid gear
- invalid mode / illegal transition

Requirements

- each fault must have a bit position
- each occurrence must increment a counter
- repeated faults across cycles must be visible in logs
- design should allow new faults to be added easily

5. Safe-State Manager

The ECU must support:

- NORMAL
- DEGRADED
- SAFE

Suggested rules

- no active faults → NORMAL
- one major active fault or repeated warnings → DEGRADED
- two or more persistent critical faults → SAFE

Suggested behaviour

- once in SAFE, system should not immediately recover unless reset condition is defined
- transitions must be deterministic
- every state change must be logged with reason

Mandatory Header File Structure

A proper header/source modular structure is required.
Teams must not submit everything in a single file.

Minimum Required Files

File	Purpose
main.c	scheduler loop and integration
input.h / input.c	input handling and validation
mode.h / mode.c	mode controller logic
control.h / control.c	overspeed, temperature, gear checks
fault.h / fault.c	fault flags, counters, fault update functions
state.h / state.c	NORMAL / DEGRADED / SAFE transitions
log.h / log.c	structured console logging

File	Purpose
types.h	common enums, structs, typedefs, macros

Header File Expectations

Each header file should contain:

- include guards
- type declarations
- macro definitions where appropriate
- function prototypes only
- no unnecessary logic inside headers

Good Practice Requirement

Common shared structures such as:

- VehicleInput
- VehicleStatus
- FaultStatus
- enums for Mode, FaultType, SystemState

should be centralized in types.h.

Required Code Skeleton

Learners must begin from or produce a codebase that follows this structure.

Suggested High-Level Skeleton

```
/* main.c */
#include "types.h"
#include "input.h"
#include "mode.h"
#include "control.h"
#include "fault.h"
#include "state.h"
#include "log.h"

int main(void)
{
    VehicleInput input = {0};
    VehicleStatus status = {0};
```

```

FaultStatus faults = {0};

init_system(&status, &faults);

while (1)
{
    read_inputs(&input);
    validate_inputs(&input, &status);

    update_mode(&status, &input, &faults);
    run_control_checks(&input, &status, &faults);
    update_fault_status(&faults);
    evaluate_system_state(&status, &faults);

    log_cycle_summary(&input, &status, &faults);
}

return 0;
}

```

Expected Supporting Functions

Teams should define functions similar to:

- init_system()
- read_inputs()
- validate_inputs()
- update_mode()
- run_control_checks()
- set_fault()
- clear_fault()
- increment_fault_counter()
- evaluate_system_state()
- log_cycle_summary()

They may refine names, but layer separation must remain clear.

Test Cases / Validation Depth

Teams must validate the system using multiple scenarios.

Mandatory Test Case 1: Normal Start

- Mode: OFF → ACC → IGNITION_ON
- Speed: 40
- Temp: 80
- Gear: 1

Expected: No faults, NORMAL state

Mandatory Test Case 2: Overspeed

- Speed: 130
- Temp: 85
- Gear: 4
- Mode: IGNITION_ON

Expected: Overspeed detected, fault counter incremented, DEGRADED or warning behaviour as defined

Mandatory Test Case 3: High Temperature

- Temp: 100

Expected: High temperature warning, no crash, correct priority handling

Mandatory Test Case 4: Critical Overheat

- Temp: 115

Expected: Critical fault, state escalation, log reason

Mandatory Test Case 5: Invalid Gear

- Gear: 9

Expected: Invalid gear fault, safe handling

Mandatory Test Case 6: Illegal Mode Transition

- OFF → IGNITION_ON directly

Expected: Illegal transition detected, FAULT mode or defined safe behaviour

Mandatory Test Case 7: Multiple Faults in One Cycle

- Speed: 140
- Temp: 120
- Gear: 9

Expected: Multiple faults set, priority respected, state escalated

Mandatory Test Case 8: Persistent Fault Across Cycles

- same critical fault repeated for 3 cycles

Expected: fault count increases, persistent behaviour visible, SAFE state reached if rule requires

Mandatory Test Case 9: Recovery / Reset Logic

- return values to normal after prior warnings

Expected: team-defined recovery behaviour documented clearly

Deliverables

1. Source Code

Complete modular source code with .c and .h files

2. File Structure Snapshot

A screenshot or text representation of the project folder structure

3. Design Note

A short document explaining:

- scheduler flow
- module responsibilities
- state transition logic
- fault handling strategy

4. Test Report

Must include:

- input scenario

- expected result
- actual result
- pass/fail observation

5. Console Logs

Representative logs showing:

- normal execution
- warnings
- faults
- state transitions

Suggested 16-Hour Breakdown

Phase	Time	Work
Architecture and file planning	3 hrs	scheduler, modules, types, headers
Core implementation	5 hrs	mode, control, input validation
Fault and state manager	3 hrs	flags, counters, transitions
Logging and integration	2 hrs	scheduler loop, outputs
Testing and debugging	2 hrs	mandatory test cases
Documentation	1 hr	design note and report